

# Design Reflection

## Architectural Decisions:

Vi valde att dela upp systemet i olika lager för att separera ansvar och skapa mer loose coupling mellan de olika delarna i programmet. Detta gjorde så att koden blev enklare att förstå och underhålla eftersom ändringar i tex Presentationslagret inte påverkade domänlogiken direkt. En nackdel med den strukturen kan vara att programmet blir något mer komplex och fler klasser kommer till, men vi ansåg att fördelarna vägde tyngre än nackdelarna.

### Presentation layer:

Presentationslagret ansvarar för användarinteraktioner och menyer. Detta lager innehåller klasser som tex Menu, ProductMenu, Productview och InputParser, som hanterar input och output till användaren men innehåller ingen affärslogik

### Application layer:

Applikationslagret innehåller serviceklasserna: ProductService, MaterialService och RecyclingService. Detta lager innehåller mer buisness logiken och fungerar som en koppling mellan användaren i presentationslagret och domänobjektet som skapas i domainlagret.

### Domain layer:

Domainlagret innehåller klasser som tex Product, Material, ProducMaterial osv. Här finns också interfacet CalculateImpact som används på olika sätt för att kunna beräkna miljöpåverkan. Det här lagret innehåller själva kärnan av systemet och gör det enklare att hålla affärsreglerna på samma ställe.

### Infrastructure layer:

Infrastructurelagret ansvarar för att lagring av data. Vi skapade ProductStorage och MemoryStorage för att kunna spara produkter som skapas i textfiler. MemoryStorage implementerar ProductStorage. Genom att göra ProductStorage till ett interface kunde vi separera lagringen från resten av programmet. Det gör att det blir mer flexibelt och lättare att underhålla, lagringslösningen kan bytas ut utan att buisness logiken behöver ändras.

## Pattern Usage:

### Strategy Pattern:

Ett designmönster som användes i projektet var Strategy Pattern. Vi använde Strategy Pattern för att kunna beräkna miljöpåverkan på flera olika sätt utan att behöva ändra större delen av koden. Genom att skapa interfacet `CalculateImpact` kunde vi enkelt lägga till nya beräkningsmetoder vid senare tillfällen om det skulle behövas. Detta gjorde koden mer flexibel och lättare att utveckla vidare, interfacet gjorde att koden blev mer extensibel och följde OCP mer. Ett annat alternativ hade varit att lägga all logik i samma klass med många if satser, men det hade gjort koden svårare att förstå och underhålla.

### Factory Pattern:

Vi använde oss även av Factory Pattern genom klassen `RecyclingGuidanceFactory`. Denna klass används för att skapa olika återvinningsinstruktioner beroende på vilken återvinningskategori produkten har. På det sättet kunde all logik för skapandet vara på ett ställe istället för att spridas ut i programmet. Det gjorde koden enklare att läsa och lättare att modifiera eller bygga ut i framtiden om fler återvinningskategorier ska läggas till.

### Interfaces och Loose Coupling:

Vi använde interfaces för att minska beroenden mellan olika klasser i systemet.

Ett exempel är att `ProductService` endast känner till interfacet `ProductStorage` och inte den konkreta implementationen av `MemoryStorage`. Detta gjorde systemet mer flexibelt eftersom sättet för lagringen kan bytas ut utan att påverka business logiken. Koden blev också enklare att underhålla och vidareutveckla.

### Validering

Validering placerades i domänlagret för att säkerställa att ogiltiga objekt inte kan skapas. Tex kontrollerar `Material`, `ProductMaterial` och `RecyclingGuidance` att värden som namn och procentandelar är giltiga. På det sättet kan vi kontrollera redan i domänlagret att affärsreglerna följs på objektnivå.

